

Untitled Fighter Game

Technical Feasibility

The project is technically $\rightarrow$ feasible because I have prior experience in Unreal Engine and C++ with a 2-3 years of experience with the Unreal Software.

Platform Selection

The Game is developed for PC because it provides powerful hardware capable of handling high-quality graphics, advanced AI, physics simulation, and detailed environments.

Engine Selection

Unreal Engine 5 was selected because it provides advanced rendering, built-in animation tools like retargeting, IK retargeting, Markerset Metahuman morph used to convert animations from a captured video. Behavior trees for AI.

Initial Risk Analysis.

Technical Risk

→ Motion warping may ~~feel~~ face some problems.

→ Animations integration ~~can~~ can face some issues in gameplay.

→ AI (Enemies) behavior should must obey the combat conditions

Eg:- When an enemy is attacking, other enemies should not attack.

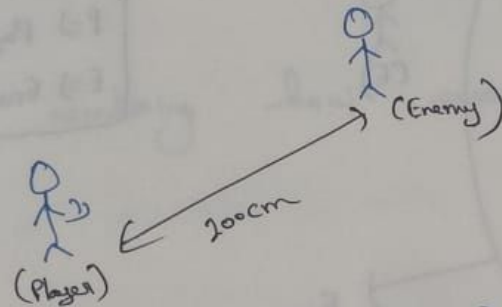
→ While activating finishers, other enemies must not attack.

→ Syncing animations in attacks and finishers will be a big task.

→ Matching the camera angle and position with the player and locked enemy (In combat)

→ High poly assets would drop performance. (Must use Nanite where appropriate) also use LODs in places where Nanite doesn't support.

NORMAL COMBAT (VS) MOTION-WARPING COMBAT



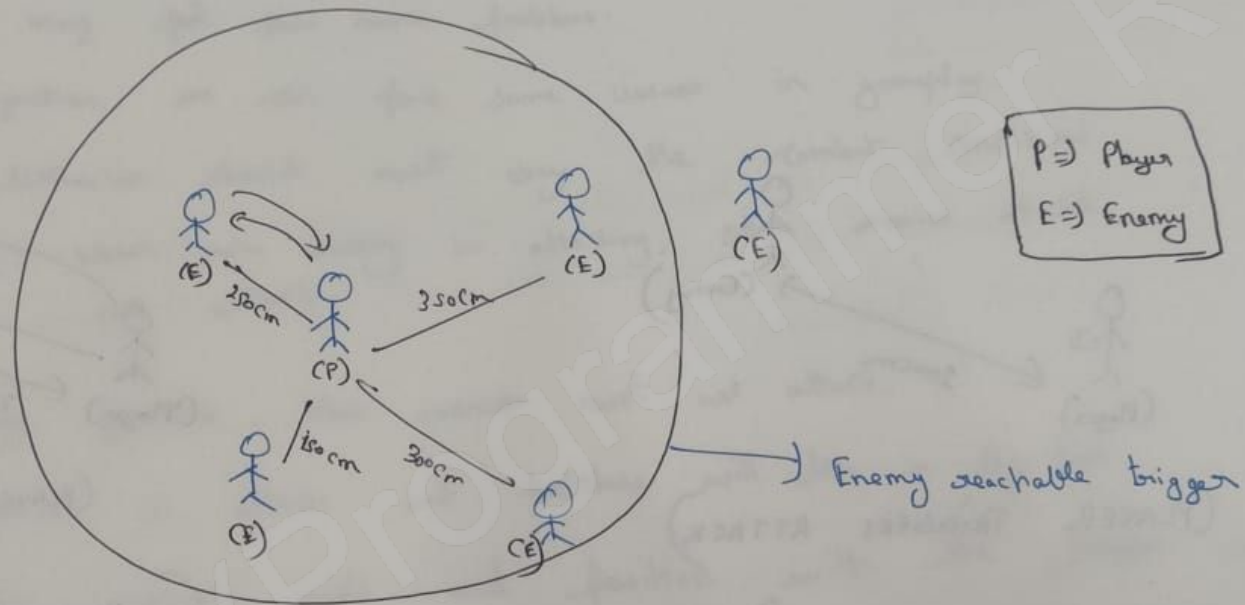
(PLAYER TRIGGERS ATTACK)

(Now Player Attacks in the same place instead of moving towards Enemy.)



(PLAYER TRIGGERS ATTACK)

(Now player ~~must~~ and enemy will face each other first, then player moves towards enemy, then attacks)



→ I was wondering how to lock to an enemy, How do the player chooses the enemy to lock on.

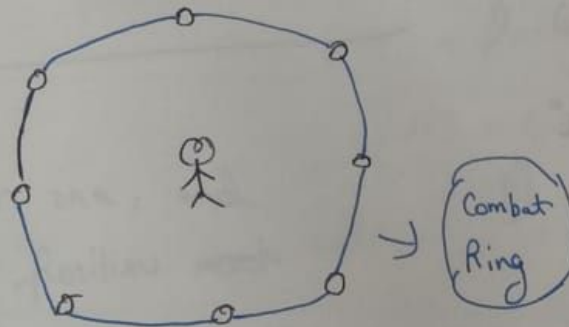
→ So then I added a USphereComponent* (Enemy reachable trigger), then add all the enemies to an array who trigger with Enemy reachable trigger.

→ Then calculating distance with all the overlapping enemies and finding the nearest enemy. Then locking on to nearest enemies.

How Combat Enemies are Positioned in Combat?

Today, I was wondering how enemies are positioned once the player enters combat.

Games like God of War, Ghost of Tsushima, Batman, Assassin and use the Combat Ring mechanic, where all enemies position them in an invisible ring around the player.



Stick figure → Player

○ → Enemies.

Combat ring Calculation

- 1) Get the No. of Enemies.
- 2) Then Find angles ~~from the~~ equal angles ^{for} the enemies enemies.

Angle $\rightarrow 0^\circ - 360^\circ$

Variables we need.

- 1) Angle difference (Must be calculated based on the no. of Enemies)
- 2) Position distance.

Example.

$\rightarrow$ Enemies $\Rightarrow 5$

$$A.O \Rightarrow 360 / 5 \Rightarrow 72^\circ$$

$$\text{Enemy 1} = 72^\circ$$

$$\text{Enemy 2} = 144^\circ$$

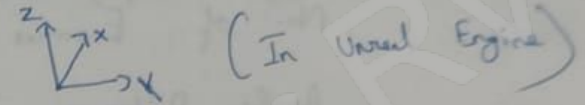
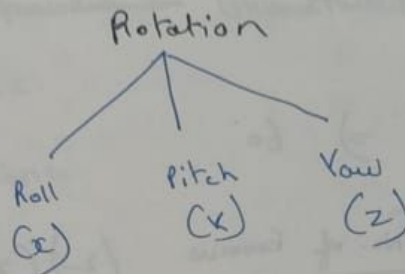
$$\text{Enemy 3} = 216^\circ$$

$$\text{Enemy 4} = 288^\circ$$

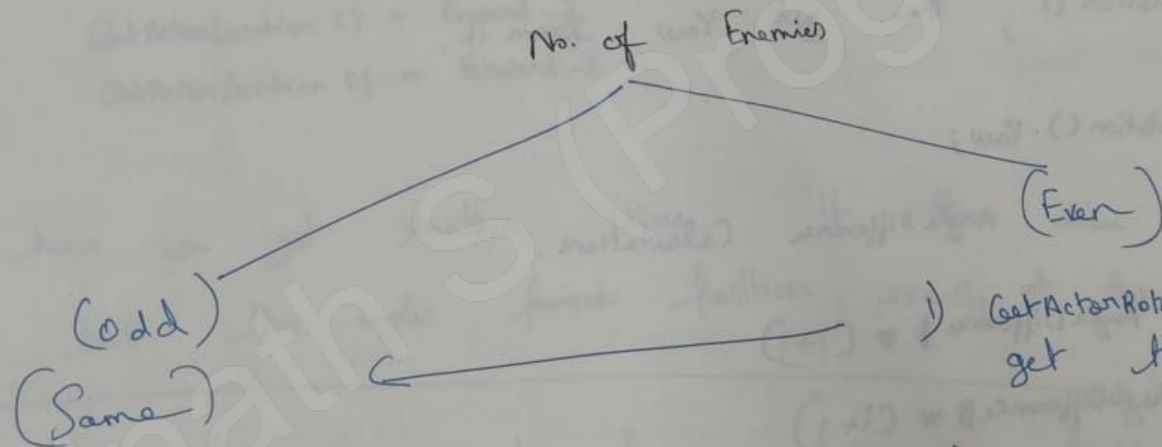
$$\text{Enemy 5} = (360^\circ \text{ (or) } 0^\circ)$$



Unreal Engine Calculation.



Now AD is calculated, with this the enemies must be positioned accordingly.



2) Decide the ~~first~~ ^{one} position one, and then decide two positions ~~next~~ in the next iteration.

- 1) GetActorRotation() → From this we need to get the yaw (z)
- 2) ~~Place~~ ^{two} Decide positions at a time.

Even No. of Enemies

Example

No. of Enemies $\Rightarrow 6$

Angle Difference $\Rightarrow 360 / 6 \Rightarrow 60^\circ$

Run a for loop 3 times (No. of Enemies / 2) (3).

```
for (int i=0; i < Enemies.num() / 2; i++)
```

Then in the iterations, we need two positions to be defined.

So, first, `GetActorRotation()`, then get Yaw from it.

```
float yaw = GetActorRotation().Yaw;
```

Then define two Yaw with Angle Difference Calculations.

```
float yaw-1 = yaw + AngleDifference * (i+1)
```

```
float yaw-2 = yaw - AngleDifference * (i+1)
```

Now create two `FRotators` with this Yaw.

```
FRotator spawnRot-1 = FRotator(GetActorRotation().Pitch, yaw-1, GetActorRotation().Roll);
```

```
FRotator spawnRot-2 = FRotator(GetActorRotation().Pitch, yaw-2, GetActorRotation().Roll);
```


Then Get Forward vectors from the spawn Rot notations.

~~Get~~ FVector Forward-1 = ~~spawnRot-1~~ UKismetMathLibrary::GetForwardVector(spawnRot-1);

Same for Forward-2.

Now Multiply it by distance.

Forward-1 * distance;

Forward-2 * distance;

6 Enemy



Then Get the position by adding Actor location & Forward vector.

GetActorLocation() + Forward-1

GetActorLocation() + Forward-2

Now here we get another issue, if there are only two enemies, then both the angles point position exactly at the back of the player.

And this logic seems to have lot of problems after testing in the Engine.

Enemy Position based on number of Enemies

Enemy = 1



2



3



4



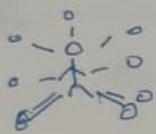
5



6



7



8



9

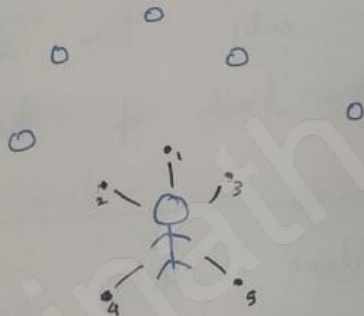


10



Logic #1

5 Enemies.



→ Now enemies will move to the position based on the distance.

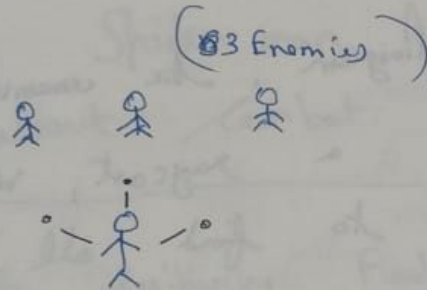
→ First position will go to the nearest enemy

→ Second position will go to the second nearest enemy.

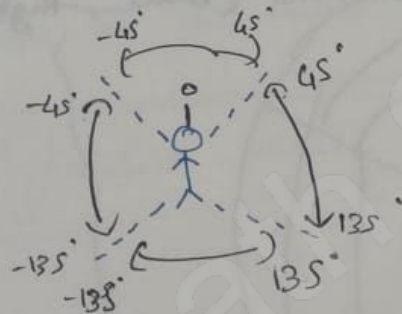
Problem In this logic.

→ Enemy in front of the player can avoid player and go to the back due to this logic.

Logic #2



#1 This logic gives priority to the forward first, then to the sides, and to the backside.



$$(360 / 4 = 90)$$

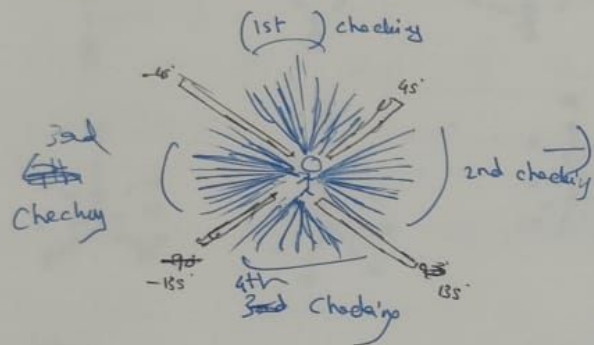
First checking → (-45° → 45°)

Second checking → (45° → 135°)

Third checking ⇒ (-45° → -135°)

Final checking ⇒ (-135° → 135°)

Now after this Logic, The enemies are picked based on visibility and distance.



① So in this diagram, the enemies are picked by drawing a raycast, we will draw 360 rays to find all enemies.

③ Now First we will start checking in the front direction, here we will shoot 90° raycast, then get all the enemies from

* ④ We have four directions (Front, Right, Left, Back)

② -45° to 45° → Front
45° to 135° → Right
-45° to -135° → Left
-135° to 135° → Back.

this, Then we will start checking in the Right, then left, and Back

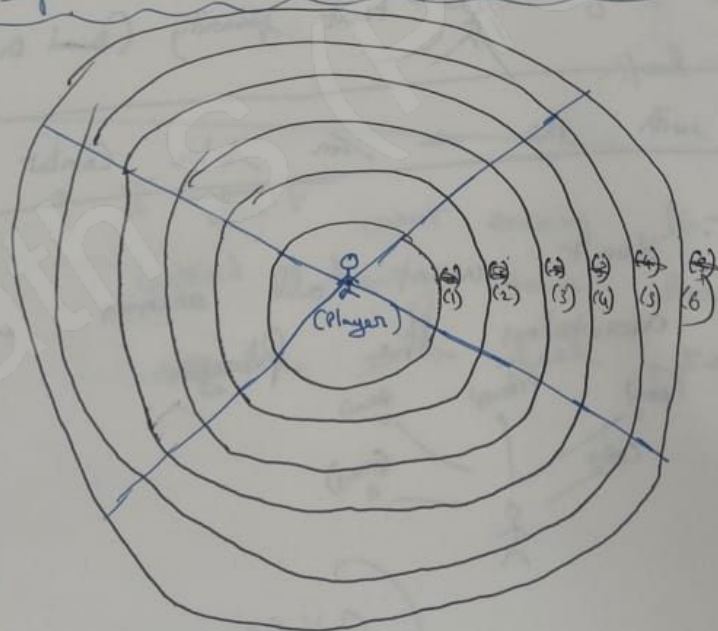
④ Why Not Left After Front?

→ Because The Game is Right side focused so keeping the Logic starting from Front and then followed by Right will give more

Clarity on enemy positioning later when we assign slots.

Now while brainstorming I got an idea on how to place enemies. After watching Spiderman AI fortnite GDC talk where they have explained about Combat Rings.

→ So to go to positioning first we must have an clarity on how we are gonna position the enemies.



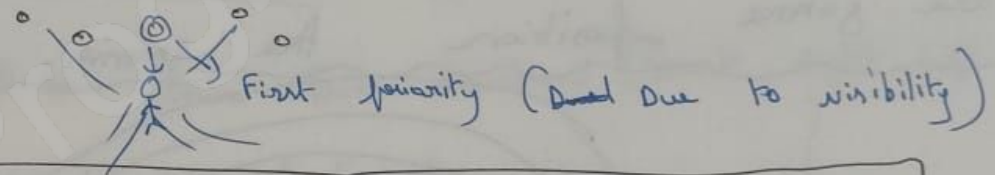
→ Now I will have 6 rings in my game where I will assign my enemies to a position in the ring.

(But where do I assign or place in the ring?)

→ So after multiple hours of brainstorming, I managed to write down my requirements.

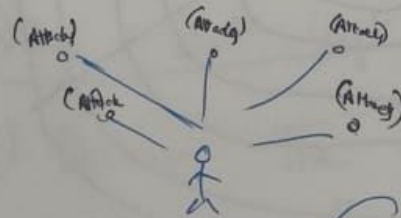
Requirements in a Beat em up free-flow Combat Game:

- ① I want the first enemy I see to ~~come and~~ ~~attack me~~ become the first priority whom the player will eventually try to attack first.



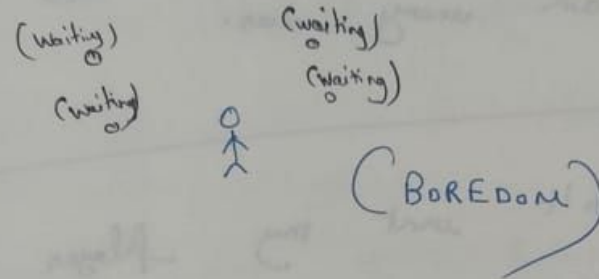
(So with this we can solve Combat Readability)

- ② Then I don't want all enemies to attack once which overwhelms the player



(OVERWHELMING)

① At the same time I don't want enemies waiting without attacking, just waiting, which feels boring.



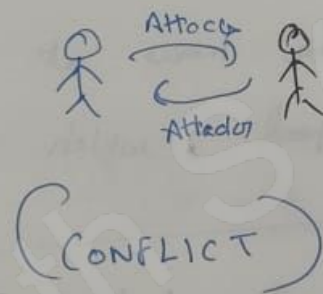
② I don't want the enemies to move in such a way that it feels artificial or natural.

for Eg. Everyone moves at the same time, same speed, which creates artificialness.

③ I don't want enemies to just crowd on to the player instead having some space and take turns and then attack will feel natural.

- Formation Collapse, I want my enemies to recalculate their slots and position them accordingly while centering the combat and while an enemy dies.

- I don't want my player ^{attacking} ~~fighting~~ and enemy attacking at the same ^{time} to make a conflict. A priority system must be created here to prevent simultaneous hits.



Problems :-

- Crowd Readability
- Slot Assignment
- Crowd Pressure
- Combat Flow
- Crowding
- Artificial Movement

So, Now my motive is to solve these problems and give engaging fast paced free flow Combat.

~~How To Solve Crowd Pressure?~~

~~How To Decide Who Must Attack?~~

Now How to design Slots.

Now To Determine ~~where to place~~ In which Ring the enemy must be placed.

Process

- We can create combat rings and position the player in the combat ring.
 - Then we create some predefined slots to place the enemies in the combat ring.
 - Then ~~make the~~ Find the justified slot for the enemy
- For example, we have 5 enemies and 5 slots, we have to create a logic in such a way that the 5 enemies must take the least distance slot.

So Now I was brain storming on how to approach this.

For Now let's say we have 5 enemies.

→ Now we need a slot placement which must ^{be} _{respective} to the player.

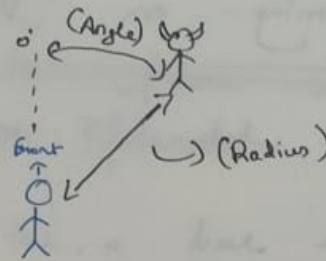
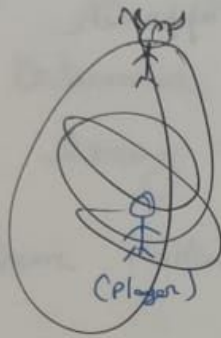
→ So to ~~sort~~ place enemy ~~to~~ _{respective} to the player, I planned ~~to~~ on getting two values, One is desired Radius and another one is desired angle.

Desired Radius :- At what distance or radius is the enemy spotted from the player.

Desired Angle :- At what angle from the player's front is the enemy positioned.

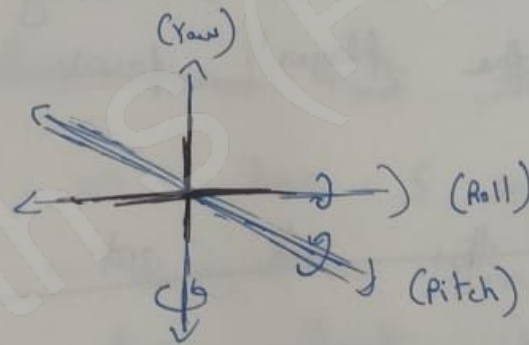
~~So Now if we take this and get~~

So Now with these two values we can get the enemy location.



So here how do we compute the location from the angle and radius.

→ First Get Actor's Rotation, and from rotation we can get the yaw angle.



→ Then we get the final angle ~~from the angle difference~~ by adding yaw angle and desired angle.

→ So here we get the angle from the player's visibility.

→ Then from this yaw angle we get forward vector.

① ~~with this~~ with this forward vector, Actor's location and radius we can get the enemy location.

② Forward Vector from Yaw & Pitch (Radiant)

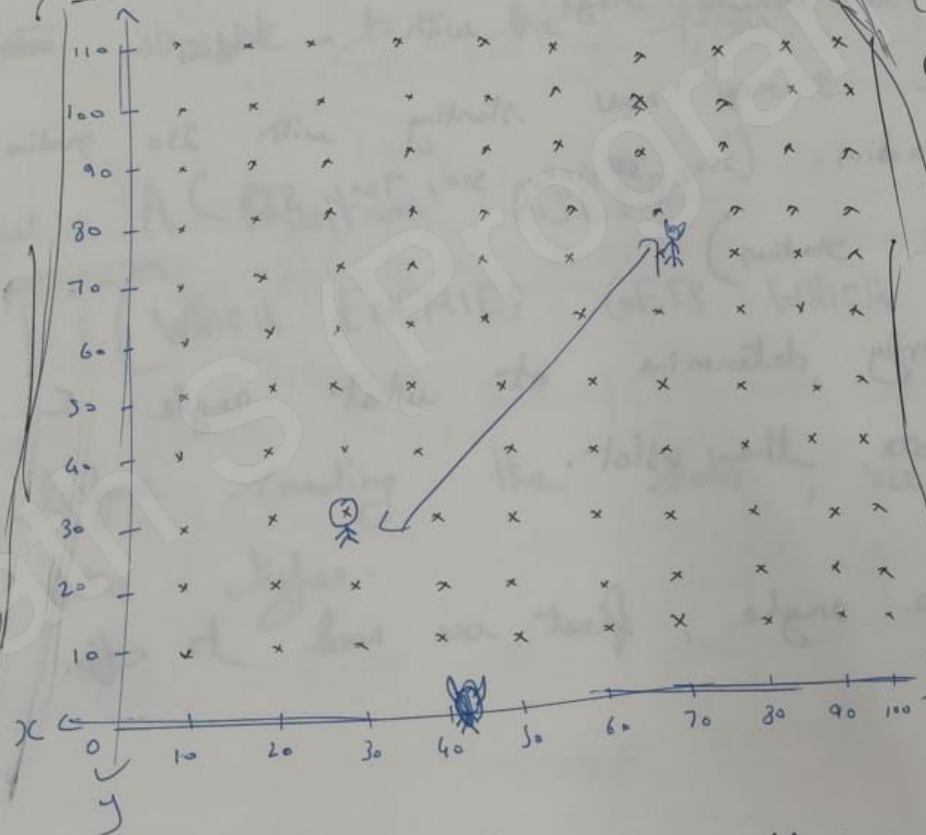
Forward Vector =

$$\begin{aligned} x &= \cos(\text{Pitch}) \cos(\text{Yaw}) \\ y &= \sin(\text{Pitch}) \\ z &= \cos(\text{Pitch}) \sin(\text{Yaw}) \end{aligned}$$

$$FR = (0.7, 0, 0.7)$$

Now we will Add
(Player's position + FR) *
Multiplied (*) by Desired
Radius.

$$\begin{aligned} (30, 30) + ((0.7, 0) * 60) \\ (30, 30) + (42, 0) \\ = (72, 30) \end{aligned}$$



Visual Representation on 2D plane.

① (PP) Player's Position :- $\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 30 \\ 30 \end{pmatrix}$

(PR) Player's Rotation :- $\begin{pmatrix} Roll & Pitch & Yaw \\ 0 & 0 & 0 \\ 0 & 45 & 27 \end{pmatrix}$

Desired Radius :- 60

Desired Angle :- 45°

$$\begin{aligned} \text{FinalAngle} &= PR.Yaw + \text{Desired Angle} \\ &= 0 + 45 = 45^\circ \end{aligned}$$

$$\begin{aligned} \text{FinalAngleInRad} &= \text{FinalAngle} * \pi / 180; \\ &= 45 * 3.14 / 180; \\ &= 0.785 \text{ radian.} \end{aligned}$$

$$\begin{aligned} \text{PitchInRad} &= PR.Pitch * \pi / 180; \\ &= 0 * 3.14 / 180 \\ &= 0 \text{ radian.} \end{aligned}$$

→ So Now I have figured out a way to create the location using angle & Radius.

→ Now I have to decide how slots will be distributed and on what basis.

→ So now First we need to create rings, with a radius.

Let's say we have 5 rings now starting with 250 radius and increasing with 150 radius. (250, 400, 550, 700, 850).

(Now we have the radius)

→ I must only determine at what angle I must create the slot.

(So to determine the angle, first we need to ~~di~~ get some ~~any~~ data)

Enemy Data

→ No. of Enemies

~~§~~

Now As I said earlier, I will have one enemy who will be locked on ~~the~~ based on player's visibility will always take 0.1 to keep the enemy ~~to~~ straight to the player.

Now A Question RISES?

(WHICH ENEMIES GETS WHICH RINGS?)

So here before creating the slots, we need to divide these enemies into types.

Inspired from God of War Combat GDC Talk

In God of War, they have divided enemy into two types.

→ Aggressive (The ones to attack)

→ Passive (Waiting for their opportunity)

Now the passive enemies are positioned a little back from the player, as their opportunity to attack is still not come.

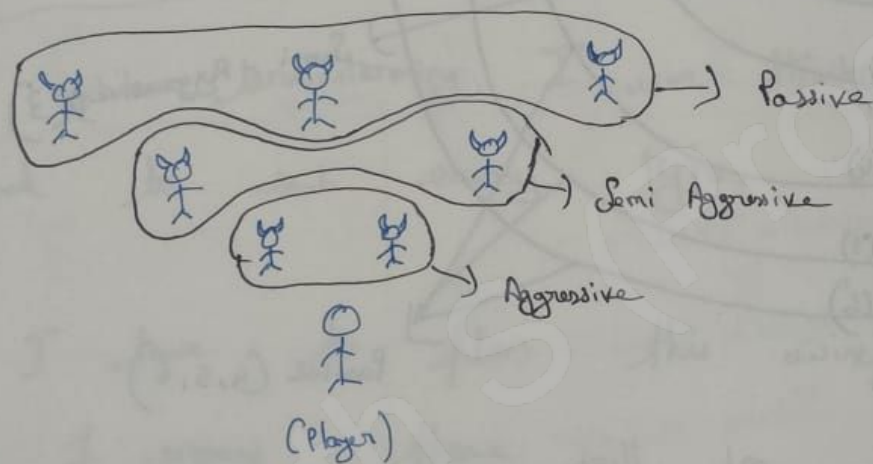
The aggressive enemies will come forward to attack the player.

Now once the attack is done, the aggressive enemies become passive, and passive becomes aggressive.

Now From this, I managed to create enemy types in my game.

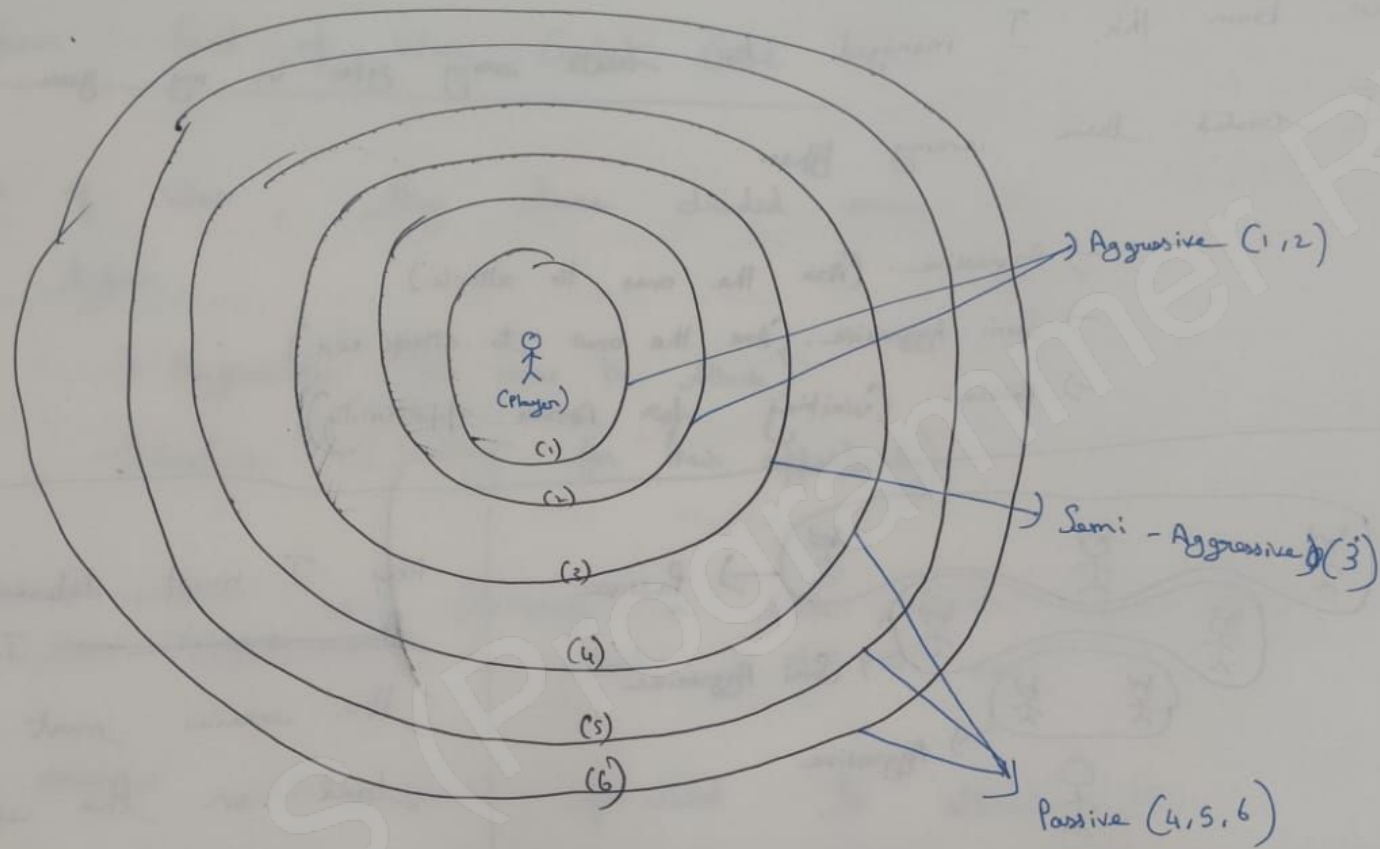
I created three enemy types.

- Aggressive. (Are the ones to attack)
- Semi Aggressive. (Are the ones to attack near)
- Passive. (Waiting for Attack opportunity)



Now I must determine ~~where~~ ~~the enemies~~ In which rings the enemies must be placed based on the enemy type.

(AGGRESSIVE — 0
SEMI - AGGRESSIVE — 1
PASSIVE — 2
)



Now I have planned to give ring 1 and 2 to aggressive enemies, as I want them to stay in first ring near to the player, as the upcoming attack is going to be performed by them.

Then I am gonna place ~~semi~~ aggressive in ring 3, ~~right back at the~~ aggressive enemies. So keeping this formation, will be easy for enemies to switch places easily, and gives combat adaptability.

Then take rest of the rings for passive enemies which waits for the attack opportunity.

Now while brainstorming, I was thinking, how many enemies must be assigned to each enemy type.

So I have to plan this wisely, because, ~~I am~~ if there is only 1 enemy, there will be no semi-aggressive and passive enemies.

Now after researching upon this, I got to know that games like God of War, Ghost of Tsushima, Sifu, ~~and~~ sleeping dogs and more use Attack Tokens to determine ~~whom~~ who should attack next.

Now for this game, we have three tokens, Aggressive, Semi-Aggressive and passive.

These tokens must be distributed based on the number of enemies.

$$\text{Aggressive Tokens} = 0$$

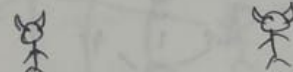
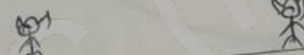
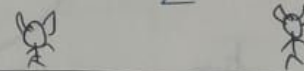
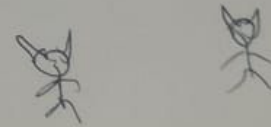
$$\text{Max Aggressive Tokens} = 2$$

$$\text{Semi-Aggressive Tokens} = 0$$

$$\text{Max Semi Aggressive Tokens} = 2$$

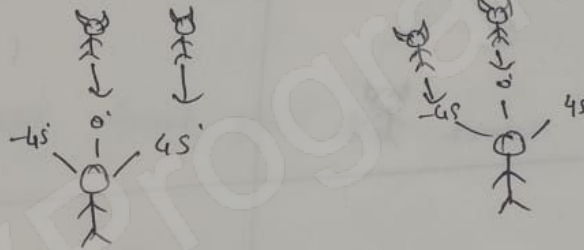
$$\text{Passive Tokens} = 0$$

$$\text{Max Passive Tokens} = \infty$$

No. of Enemies	Aggressive	Semi-Aggressive	Passive
	1 	0	0
	2 	0	0
	2 	1 	0
	2 	2 	0
	2 	2 	1 
	2 	2 	2 

Now I have to create angles for the enemies based on the constraint.

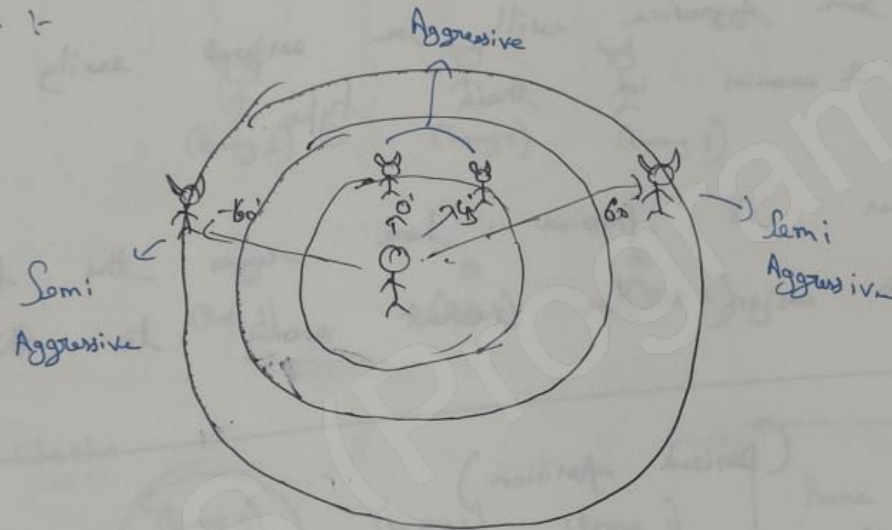
For Aggressive type enemies, we can have only three angles ~~less~~



As we have only two enemies in Aggressive, The First enemy who is locked on will take the 0° degree, then ~~the~~ other aggressive will be given -45° or 45° based on nearest distance.

Then for the semi aggressive, we place them in Ring 3, will be split apart a little bit more for combat readability.

Here I am planning to place them in -60° and 60° which will form:-



And all other enemies will be scattered in ^{Rings} ~~Rings~~ 4, 5 & 6, who are termed as "Passive".

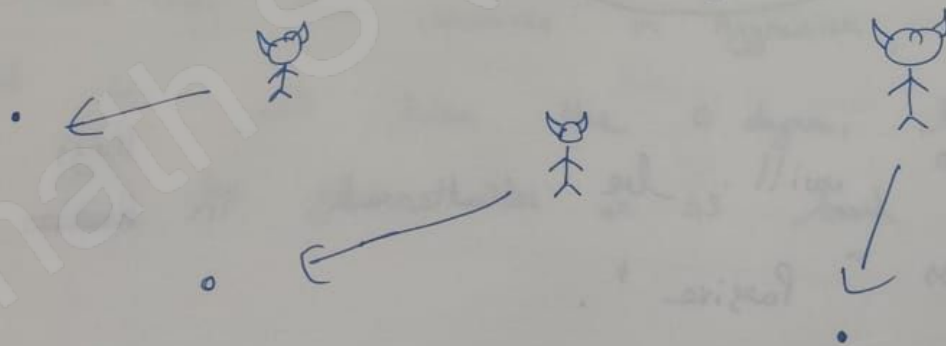
Now once the angle and rings are ready we will start assigning those slots to the enemies.

Aggressive & semi aggressive will be assigned easily as we have a max of 2 enemies in their types.

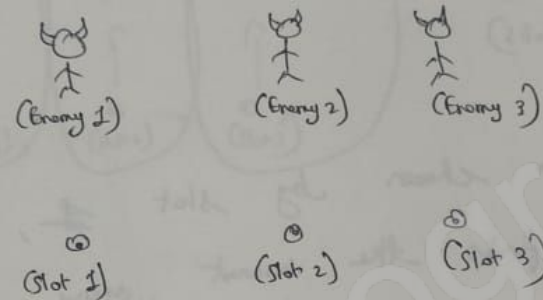
→ When it comes to passive, here comes the problem, Now I have to assign the nearest enemy to this slot.

Logic #1

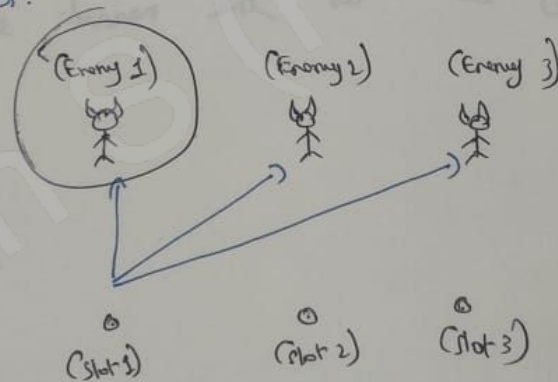
(Desired operation)



So now in this logic I will iterate through each slot and find a nearest enemy which will get the desired enemy for the slot



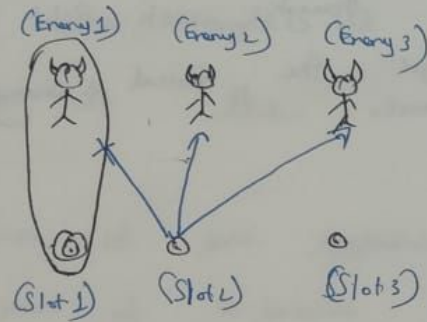
Now Slot 1 checks.



Here, the slot 1 checks with all the three enemies, and finds the nearest enemy.

Now In this we see enemy 1 is the nearest.

Now slot 2 checks.



Now as the enemy 1 is chosen by slot 1, the slot 2 checks other two enemies. Then finds the nearest enemy in these.

Now slot 2 gets enemy 2 as the nearest enemy.

Now Slot 3 checks

